

2026년 인공지능 선도사업  
서울권역  
AICOSS 연합 경진대회

2026. 07. 13 (월) ~ 2026. 07. 15 (수)



서울시립대학교  
UNIVERSITY OF SEOUL



성균관대학교  
SUNGKYUNKWAN UNIVERSITY



서울과학기술대학교  
SEOULTECH SEOUL NATIONAL UNIVERSITY OF SCIENCE & TECHNOLOGY

## 이재호 단장님

서울시립대학교 인공지능혁신융합대학 사업단장

서울시립대학교 전기전자컴퓨터공학부 교수

**ISO/IEC TS 6254** 인공지능분야 프로젝트 리더

**KS** 정보산업부회 전자문서처리언어 전문위원회 위원장





노영민 교수



채승혜



문승재



조찬슬

| 날짜     | 1/8 (수)                                                                                                            | 1/9 (목)                                                                                  | 1/10 (금)                                                                                                                             |
|--------|--------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| 10:00~ | 개회식 (30분)<br>오리엔테이션 (90분)                                                                                          | 이론 및 구현 실습 (120분) <ul style="list-style-type: none"> <li>Multi-head attention</li> </ul> | 이론 및 구현 실습 (120분) <ul style="list-style-type: none"> <li>Vision Foundation Models</li> </ul>                                         |
| 12:00~ | 점심식사 (90분)                                                                                                         | 점심식사 (90분)                                                                               | 점심식사 (90분)                                                                                                                           |
| 13:30~ | 이론 강의 및 구현 실습 (120분) <ul style="list-style-type: none"> <li>token</li> <li>embedding</li> <li>attention</li> </ul> | 이론 및 구현 실습 (120분) <ul style="list-style-type: none"> <li>Transformer의 구성요소</li> </ul>    | 이론 및 구현 실습 (120분) <ul style="list-style-type: none"> <li>Large Language Models</li> </ul>                                            |
| 15:30~ | 이론 강의 및 구현 실습 (60분) <ul style="list-style-type: none"> <li>self-attention</li> </ul>                               | 이론 및 구현 실습 (60분) <ul style="list-style-type: none"> <li>Transformer 구현 실습</li> </ul>     | 이론 및 구현 실습 (120분) <ul style="list-style-type: none"> <li>Vision Language Models</li> <li>Multimodal Large Language Models</li> </ul> |

## 1일차

- 서버 및 주피터 환경
- Pytorch 와 Tensor 실습
- Token 과 Embedding
- Attention / Self-attention mechanism 이론 및 구현 실습

## 2일차

- Multi-head attention 이론 및 구현 실습
- Transformer
  - Transformer 의 구성 (Positional Encoding, LayerNorm, FFN, Residual Connection)
  - Transformer 구현 실습

## 3일차

- Vision Foundation Models
- Pre Training 과 Transfer Learning
- Large Language Models
- Vision Language Models
  - CLIP (Contrastive Language Image Pre Training)
  - Multi-modal Large Language Models (MLLM)





※식사는 팀 단위로 식사하며, 배부된 식권은 경진대회 기간 내에만 사용 가능합니다.

|    | 팀원                                            |
|----|-----------------------------------------------|
| 1팀 | 정이주(서울과기대), 김성우(서울시립대), 전유빈(성균관대)             |
| 2팀 | 김민우(서울시립대), 강수진(서울과기대), 양대운(성균관대)             |
| 3팀 | 신승우(서울과기대), 이채원(성균관대), 박성현(서울과기대)             |
| 4팀 | 이혜연(성균관대), 박종혁(서울시립대), 조강현(서울과기대)             |
| 5팀 | 박현우(서울시립대), 노유정(서울과기대), 이석훈(서울과기대)            |
| 6팀 | 김세훈(서울과기대), 정재홍(서울시립대), 김채현(서울과기대)            |
| 7팀 | 강은수(서울시립대), 김윤수(성균관대), 허정민(서울과기대)             |
| 8팀 | 김건우(성균관대), 조재희(서울시립대), 안여은(서울과기대), 김민재(서울과기대) |

※ 팀 배정은 카카오톡 오픈채팅방을 통해 전달 드린 구글폼 설문조사에 기반하여  
배정되었습니다.

## 팀장 선출 및 팀이름 정하기 (10분)

|    | 팀명 | 팀장 | 팀원                                               |
|----|----|----|--------------------------------------------------|
| 1팀 |    |    | 정이주(서울과기대), 김성우(서울시립대), 전유빈(성균관대)                |
| 2팀 |    |    | 김민우(서울시립대), 강수진(서울과기대), 양대운(성균관대)                |
| 3팀 |    |    | 신승우(서울과기대), 이채원(성균관대), 박성현(서울과기대)                |
| 4팀 |    |    | 이혜연(성균관대), 박종혁(서울시립대), 조강현(서울과기대)                |
| 5팀 |    |    | 박현우(서울시립대), 노유정(서울과기대), 이석훈(서울과기대)               |
| 6팀 |    |    | 김세훈(서울과기대), 정재홍(서울시립대), 김채현(서울과기대)               |
| 7팀 |    |    | 강은수(서울시립대), 김윤수(성균관대), 허정민(서울과기대)                |
| 8팀 |    |    | 김건우(성균관대), 조재희(서울시립대), 안여은(서울과기대),<br>김민재(서울과기대) |

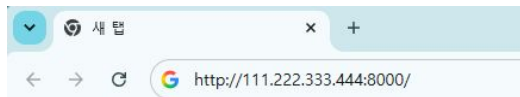
※ 팀 배정은 카카오톡 오픈채팅방을 통해 전달 드린 구글폼 설문조사에 기반하여  
배정되었습니다.





## 서버 사용법

- 브라우저에 지급받은 <본인 서버 주소>입력

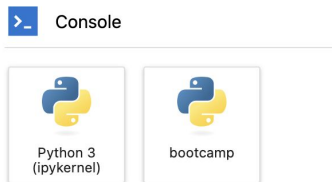


- Password or token 칸에 `bootcamp2026` 입력 후 Log in

Password or token:

# 강의자료 다운로드

0. jupyter lab의 console python3(ipkernel) 접속



1. 하단 빈칸에 아래 내용 입력하고 shift+enter

```
[ ]: !git clone https://github.com/SeunghyeChae/aicoss-0713.git
```

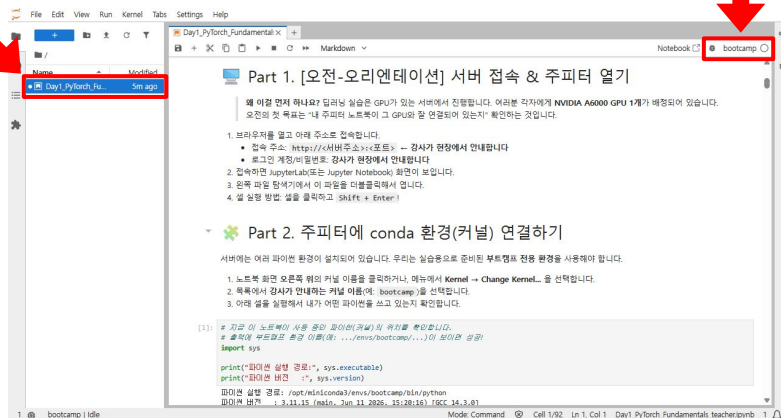
2. 왼쪽 폴더 - 파일 목록에서 .ipynb 형식의 강의 실습 파일 열기

| /           |          |
|-------------|----------|
| Name        | Modified |
| aicoss-0713 | next yr. |

3. 우측 상단 커널이 bootcamp 인지 확인 (아니면 클릭해서 변경)

- + 일어날 수 있는 문제 (커널 점유, 종료 방법 등) 알려주기

강의 실습  
파일



가상환경  
(커널)

## 서버이용시 참고사항

1. 실습 중간중간 파일을 저장해주세요.
2. 실습이 끝나면 커널을 종료해주세요
  - `Kernel` → `Shut Down Kernel`
3. 브라우저만 종료하면 커널은 계속 실행될 수 있습니다.
  - 반드시 커널 또는 서버를 종료한 후 종료해주세요.
4. 실행이 되지 않는 경우
  - 우측 상단의 커널이 **bootcamp**인지 확인
  - `Kernel` → `Restart Kernel` 실행
  - 그래도 해결되지 않으면 운영진에게 문의
5. 셀 실행이 계속 멈춰있는 경우
  - 다른 셀이 실행 중인지 확인
  - `Kernel` → `Interrupt Kernel` 또는 `Restart Kernel`을 사용하세요.
  - 패키지 설치(**pip install**)는 금지합니다.
6. 필요한 라이브러리는 미리 설치되어 있습니다.
  - 서버 과부하 방지를 위해 불필요한 **Notebook**, **Console**은 종료해주세요.

## 부트캠프 규칙

1. **GPT, Claude, Gemini 등 LLM(생성형 AI) 사용을 금지**합니다.
  - 적발 시 팀 감정 요인이 될 수 있습니다.
2. **강의 시작 시간**을 반드시 준수합니다.
  - 지각 시 불이익이 발생할 수 있습니다.
3. **팀원과 적극적으로 소통하며 협업**합니다.
  - 역할을 분담하고 서로를 존중해주세요.
4. **부정행위를 금지**합니다.
  - 타 팀의 결과물을 무단으로 사용하거나 공유받는 행위를 금지합니다.
  - 제공된 서버 계정은 본인만 사용합니다.
5. 안전사고 예방을 위해 행사장 내 안전수칙을 반드시 준수합니다.
  - 비상상황 발생 시 운영진의 안내를 따라주세요.
6. 촬영된 사진 및 영상은 행사 홍보 목적으로 활용될 수 있습니다.
7. 문의사항이나 문제가 발생하면 즉시 운영진에게 알려주세요.
8. 모두가 즐겁고 공정하게 참여할 수 있도록 기본적인 예의를 지켜주세요.

# From Tensor Operation to Transformer

2026년 인공지능 컨소시엄 서울권역  
AICOSS 연합 경진대회



서울시립대학교  
UNIVERSITY OF SEOUL



성균관대학교  
SUNGKYUNKWAN UNIVERSITY



서울과학기술대학교  
SEOULTECH SEOUL NATIONAL UNIVERSITY OF SCIENCE & TECHNOLOGY

# Preliminary

# 토큰 (Token)

## 토큰(Token) 이란 ?

모델이 문장을 처리하기 위해 나누는 기본 단위

이 토큰들은 컴퓨터가 읽을 수 있는 고유한 번호(Token ID)로 바뀜

ex1) 문장: I love artificial intelligence.

→ 토큰: [ “I”, “love”, “artificial”, “intelligence”, “.” ] → [34, 521, 1824, 3967, 13]

ex2) 문장: 나는 인공지능을 공부한다.

→ 토큰: [ “나는”, “인공지능”, “을”, “공부”, “한다”, “.” ] → [34, 521, 1824, 3967, 13]



# 벡터와 임베딩 (Vector and Embedding)

## 임베딩 (Embedding)이란 ?

토큰을 컴퓨터가 이해할 수 있는 숫자 벡터(Vector)로 변환한 표현

ex1) 토큰: "dog"

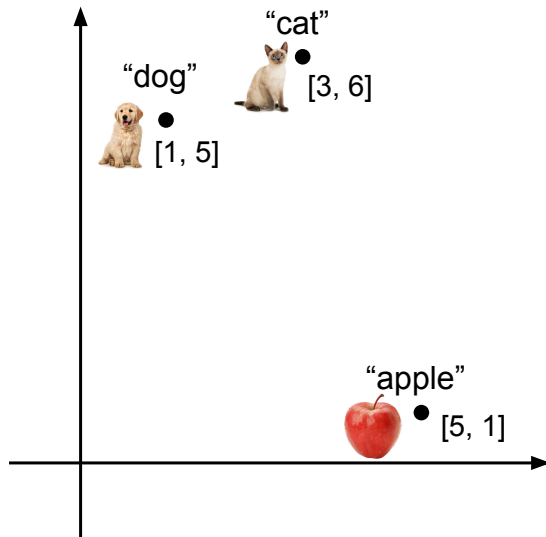
→ 임베딩: [1, 5] (Channel = 2)

ex2) 토큰: "날씨"

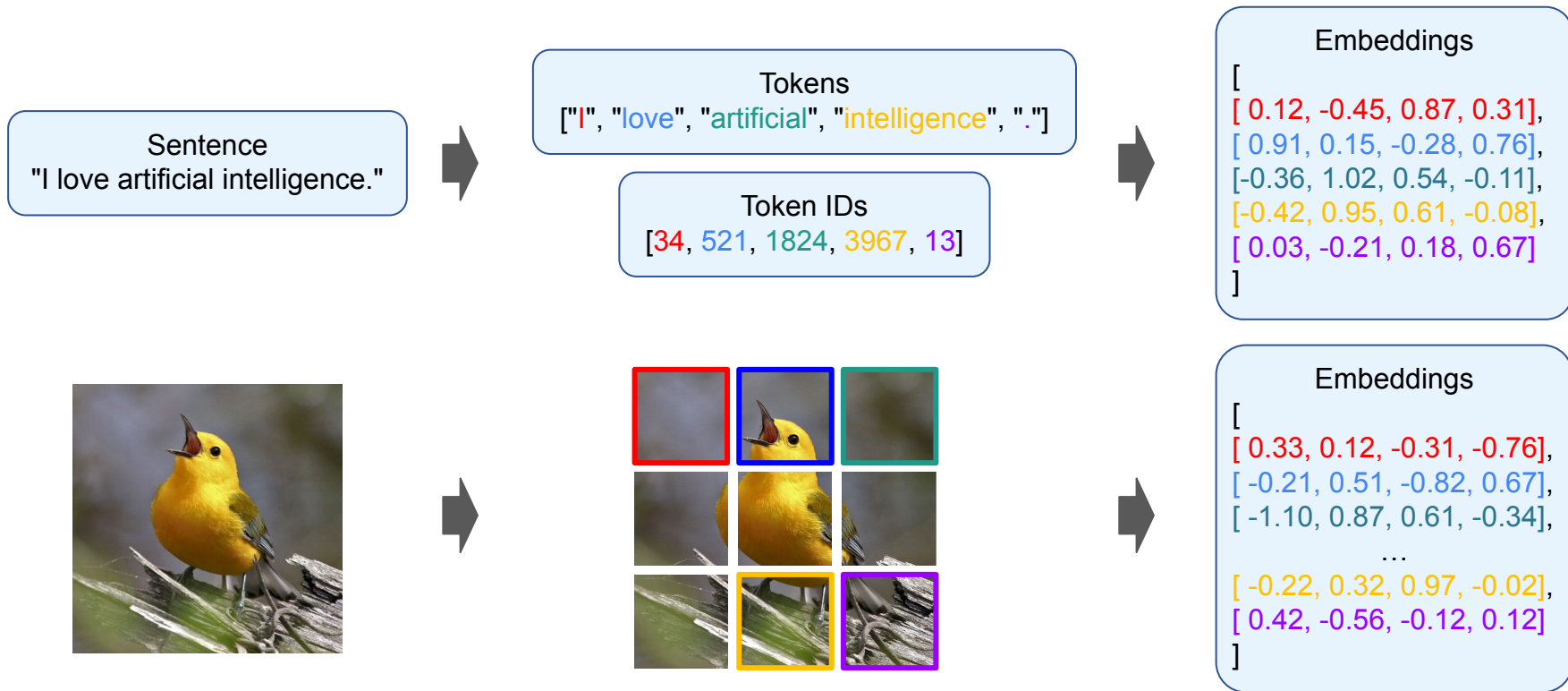
→ 임베딩: [0.12, -0.45, ..., 0.08] (Channel = n)

## 임베딩의 특징

- 고차원 벡터 공간에서 단어의 의미를 표현
- 의미가 비슷한 단어는 벡터 공간에서 가까움
- Channel 이 클수록 더 많은 숫자 표현을 할 수 있어, 더 자세한 정보를 담을 수 있음



# Sentence / Image → Tokens / Patches → Embedding



# Attention

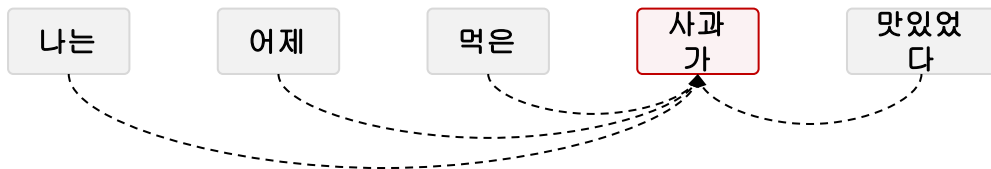
# Attention Intuition

어떤 단어가 다른 단어들을 참고할 때, “어떤 정보를 얼마나 중요하게 볼지” 결정하는 과정

예시 문장

“나는 어제 먹은 사과가 맛있었다”

(‘사과’를 이해하려고 할 때)



모든 단어가 동일하게 중요하지 않음



**Attention = 어떤 정보를 얼마나 참고할지 결정**

# Query(Q), Key(K), Value(V)



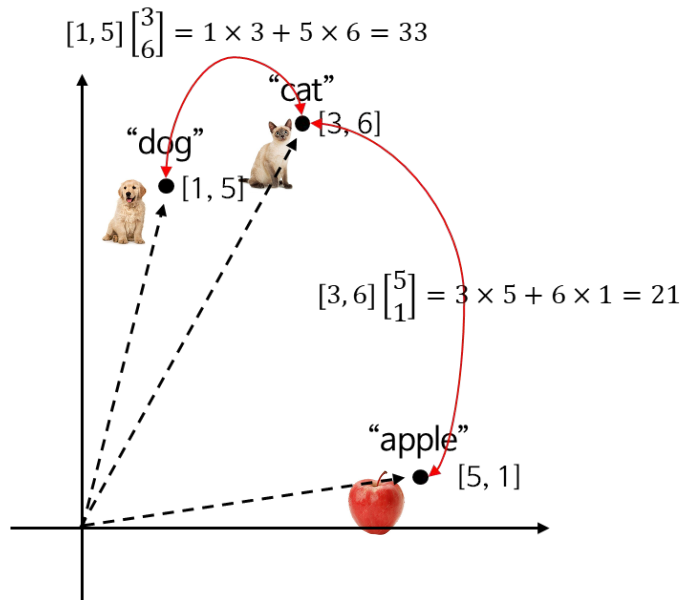
Query 와 Key 의 유사도(관련도)를 계산하고  
유사도가 높을수록 해당 Value 를 더 많이 반영

## Query(Q) - Key(K) Matching (중요도 계산)



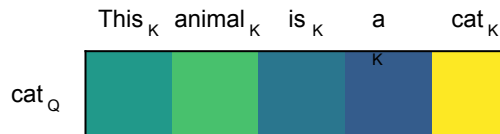
Query 와 Key 의 유사도(관련도)를 계산하고  
유사도가 높을수록 해당 Value 를 더 많이 반영

# Similarity Score (Attention Score)

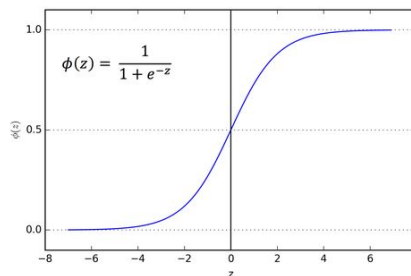


문장 예시:

This animal is a cat



$$\text{Attention Score} = \text{softmax}\left(\frac{QK^T}{\sqrt{d}}\right)$$



점수를 확률처럼 바꿔주는 함수

“Softmax”

Softmax는 여러 점수 중 중요한 값은 더 크게, 덜 중요한 값은 더 작게 만들어 전체 합이 1이 되도록 변환하는 함수이다.

Attention에서는 각 토큰이 다른 토큰을

얼마나 참고할지 점수로 계산한 뒤,

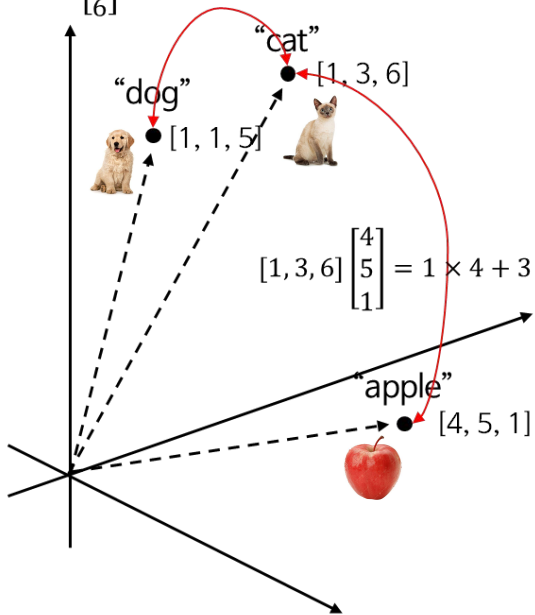
Softmax를 통해 그 점수들을 ‘비율’로

바꾼다. 이 비율이 Value에 곱해져

최종적으로 어떤 정보를 더 많이 가져올지 결정한다.

# Similarity Score (Attention Score)

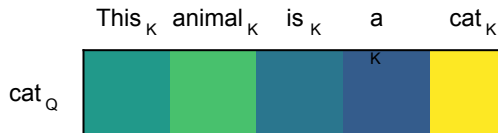
$$[1, 1, 5] \begin{bmatrix} 1 \\ 3 \\ 6 \end{bmatrix} = 1 \times 1 + 1 \times 3 + 5 \times 6 = 34$$



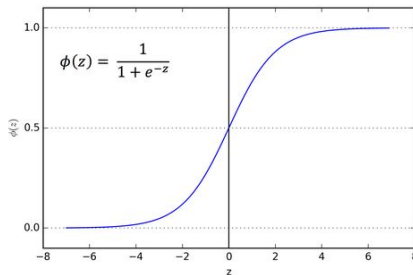
$$[1, 3, 6] \begin{bmatrix} 4 \\ 5 \\ 1 \end{bmatrix} = 1 \times 4 + 3 \times 5 + 6 \times 1 = 25$$

문장 예시:

This animal is a cat



$$\text{Attention Score} = \text{softmax}\left(\frac{QK^T}{\sqrt{d}}\right)$$



점수를 확률처럼 바꿔주는 함수

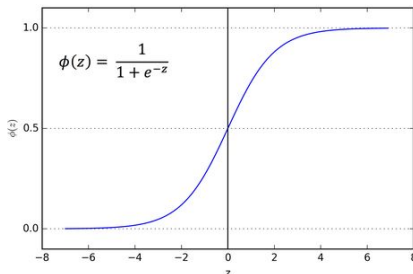
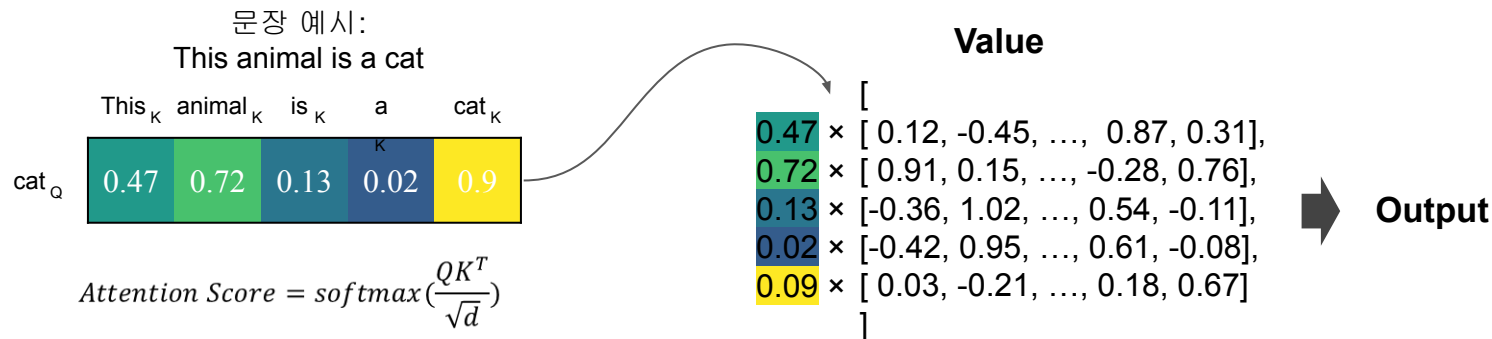
“Softmax”

Softmax는 여러 점수 중 중요한 값은 더 크게, 덜 중요한 값은 더 작게 만들어 전체 합이 1이 되도록 변환하는 함수이다.

Attention에서는 각 토큰이 다른 토큰을 얼마나 참고할지 점수로 계산한 뒤, Softmax를 통해 그 점수들을 ‘비율’로 바꾼다. 이 비율이 Value에 곱해져 최종적으로 어떤 정보를 더 많이 가져올지 결정한다.



# Attention Scores as Weights for Values



점수를 확률처럼 바꿔주는 함수

“Softmax”

Softmax는 여러 점수 중 **중요한 값은 더 크게, 덜 중요한 값은 더 작게** 만들어 전체 합이 1이 되도록 변환하는 함수이다.

Attention에서는 각 토큰이 다른 토큰을 얼마나 참고할지 점수로 계산한 뒤, Softmax를 통해 그 점수들을 ‘비율’로 바꾼다. 이 비율이 Value에 곱해져 최종적으로 어떤 정보를 더 많이 가져올지 결정한다.

## Query(Q) - Key(K) Matching (중요도 계산)



Query 와 Key 의 유사도(관련도)를 계산하고  
유사도가 높을수록 해당 Value 를 더 많이 반영

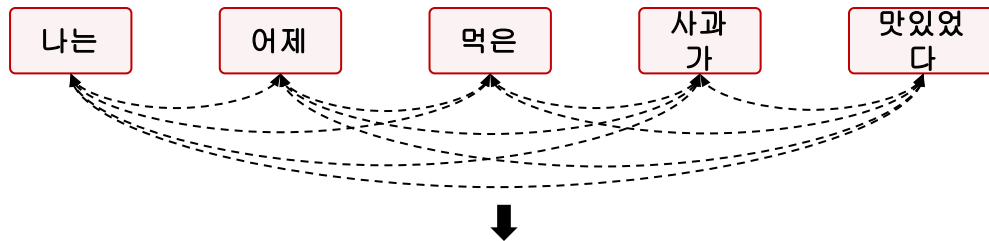
# Self-Attention

## Query-Key Matching in Self-Attention

◎ 문장 내 모든 단어가 다른 모든 단어를 참고하여 전체 문맥을 이해

예시 문장

“나는 어제 먹은 사과가 맛있었다”



**Self-Attention** = 자기 자신 문장 내에서 모든 단어가 각 단어 정보를 얼마나 참고할지 결정하여 전체 문맥에 대해 이해

# Query-Key Matching in Self-Attention



Query 와 Key 의 유사도(관련도)를 계산하고  
유사도가 높을수록 해당 Value 를 더 많이 반영

## Query-Key Matching in Self-Attention



Query 와 Key 의 유사도(관련도)를 계산하고  
유사도가 높을수록 해당 Value 를 더 많이 반영

# Query-Key Matching in Self-Attention



Query 와 Key 의 유사도(관련도)를 계산하고  
유사도가 높을수록 해당 Value 를 더 많이 반영

# Query-Key Matching in Self-Attention



Query 와 Key 의 유사도(관련도)를 계산하고  
유사도가 높을수록 해당 Value 를 더 많이 반영

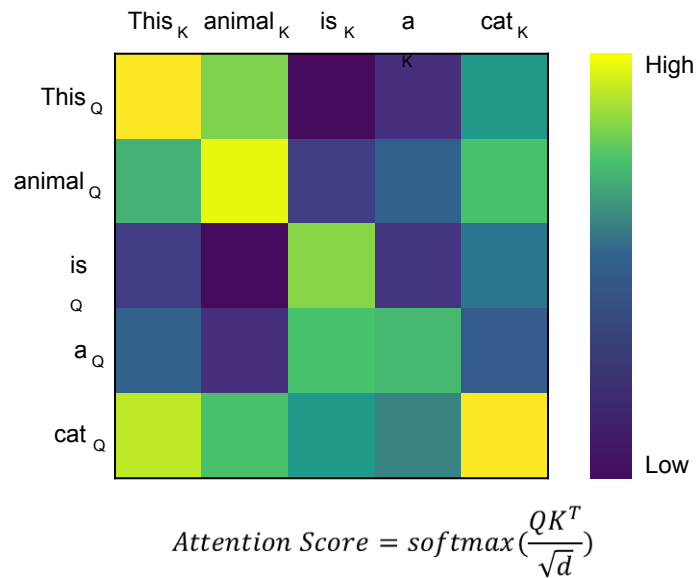
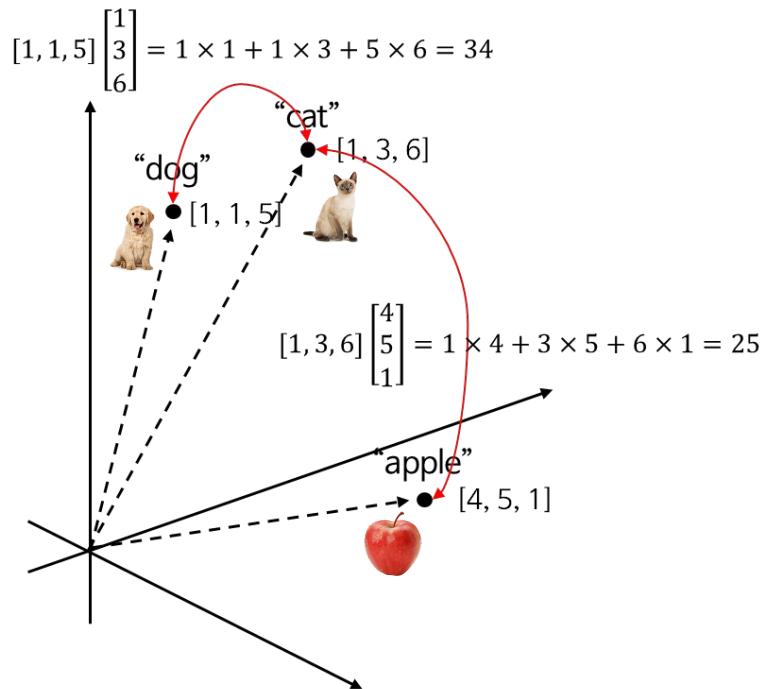


# Query-Key Matching in Self-Attention

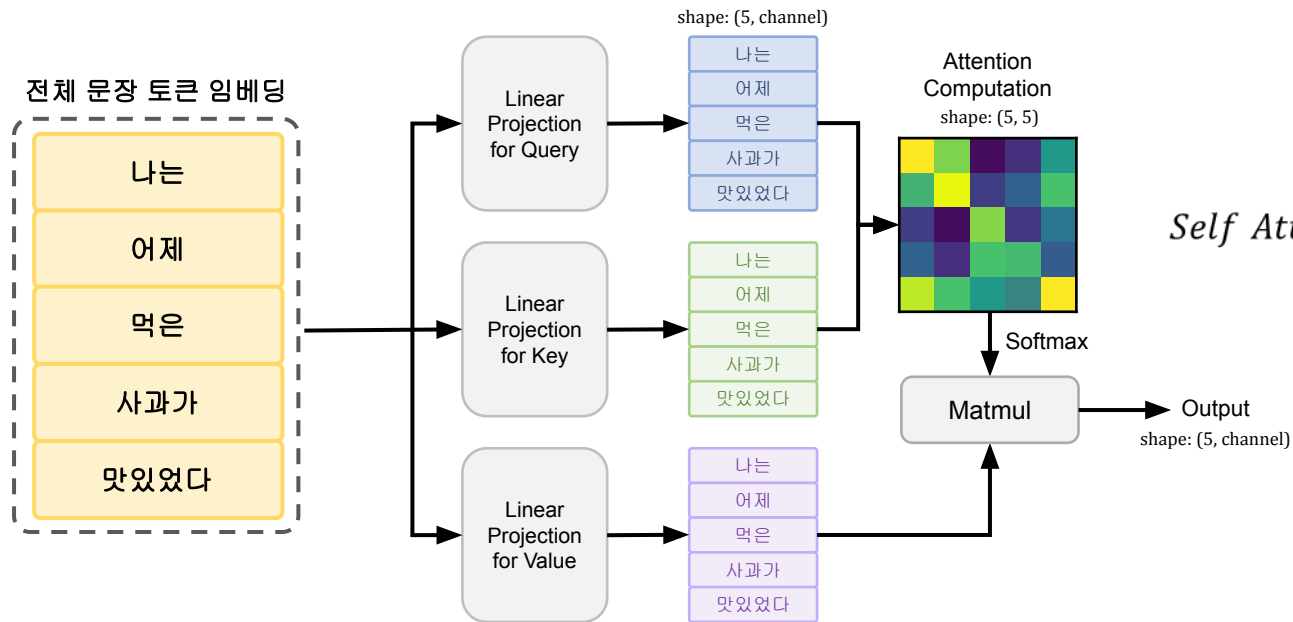


Query 와 Key 의 유사도(관련도)를 계산하고  
유사도가 높을수록 해당 Value 를 더 많이 반영

# Parallel Operation of Self-attention Computation



# Full Self-attention Computation



$$Self\ Attention = softmax\left(\frac{QK^T}{\sqrt{d}}\right)V$$

# Multi-head Attention

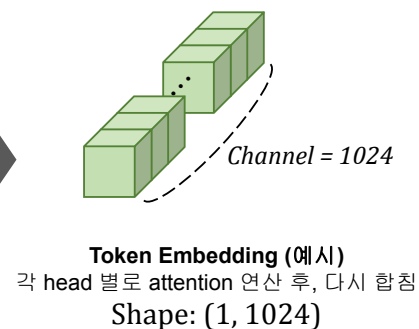
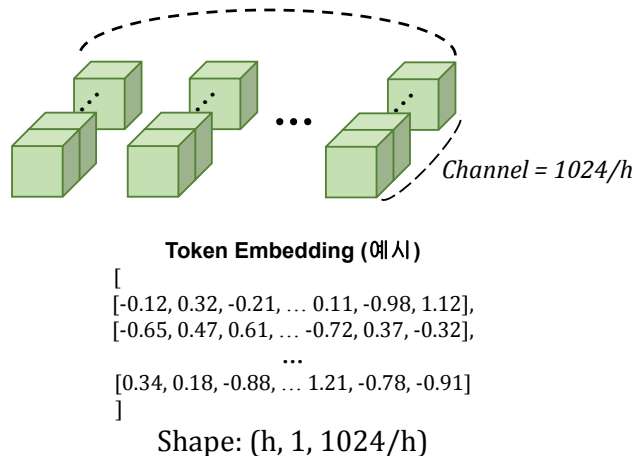
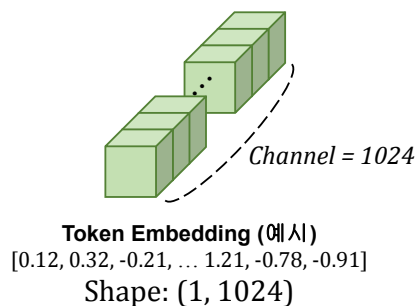
# Multi-head Attention

## ※ Intuition

여러개의 전문가(Attention Head)가 서로 다른 관점(channel)을 맡아 관계를 학습

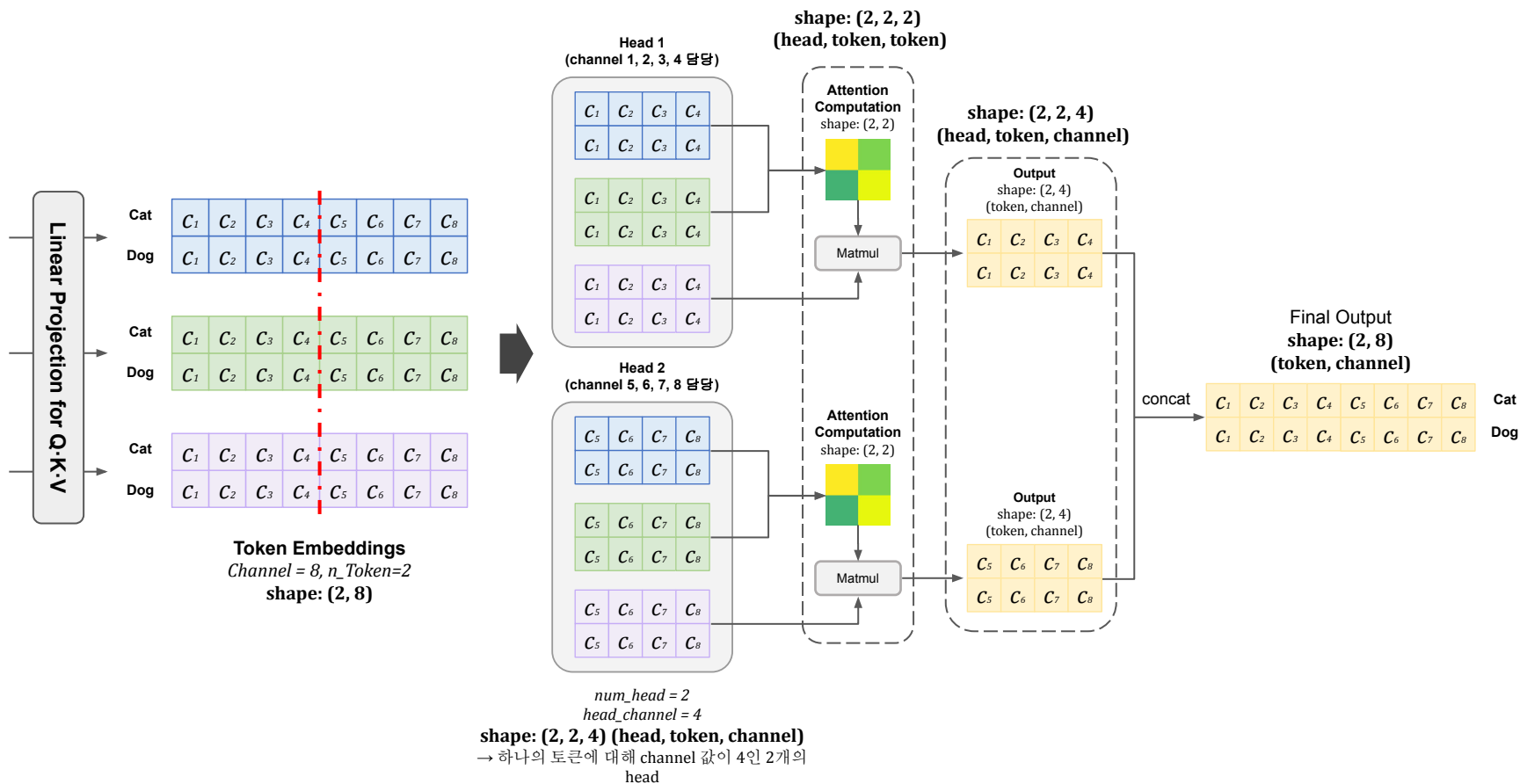
(여러 관점에서의 정보를 결합하여 더 풍부한 정보를 획득)

$h$  개의 head



→ 하나의 토큰을  $1024/h$  개의 값으로 이루어진  $h$  개의 벡터 임베딩으로 표현

# Multi-head Attention (Example)

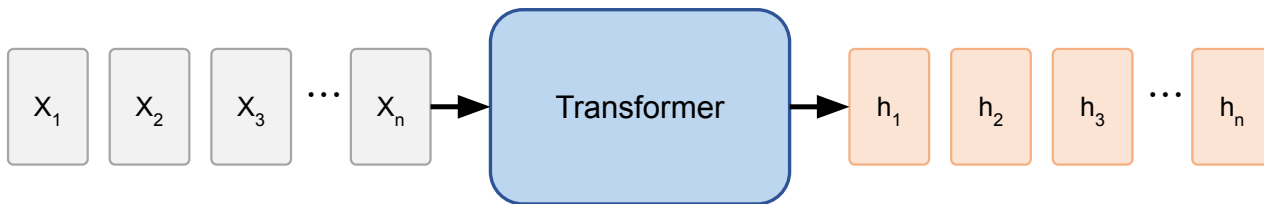


# Transformer

# Transformer

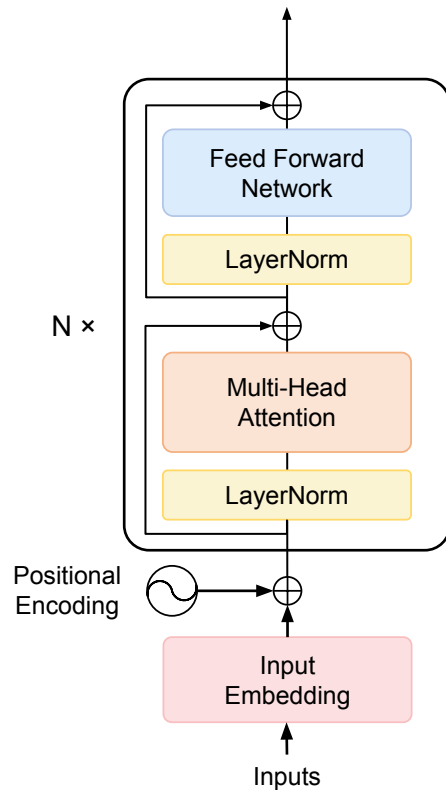
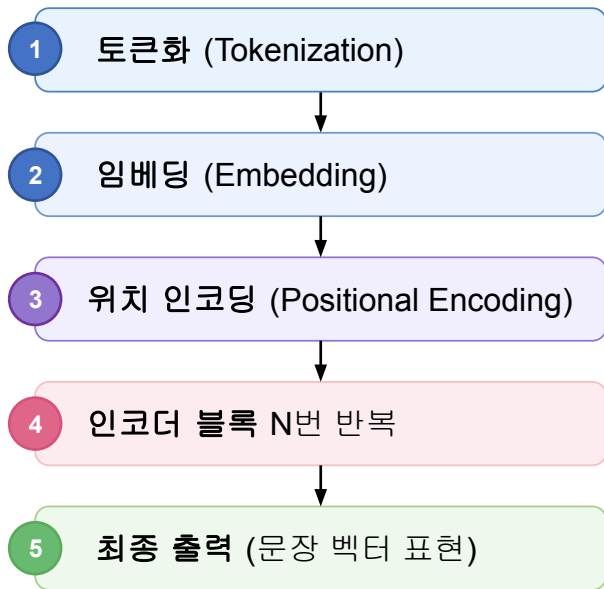
Transformer는 Self-Attention을 통해 입력 간 관계를 효과적으로 학습하는 현대 AI의 핵심 아키텍처

- ✓ Self-Attention 메커니즘 사용
- ✓ RNN과 달리 순차 처리 없이 전체 입력을 동시에 병렬 처리
- ✓ 강한 장거리 의존성 학습
- ✓ 현대 LLM 및 Vision Architecture 들의 핵심 기술





# Transformer Block



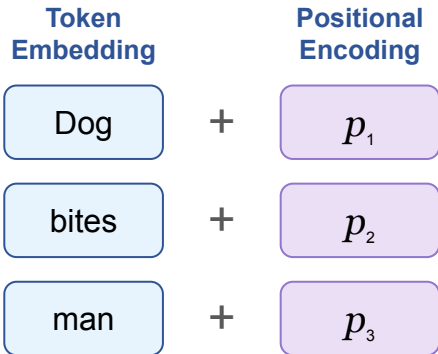
## 위치 인코딩 (Positional Encoding)

Transformer 는 self-attention 을 통해 단어에 대한 내용을 잘 알지만,  
순서 정보가 주어지지 않으면 문장 정보를 알기 어려움

“Dog bites man.” ↔ “Man bites dog.”

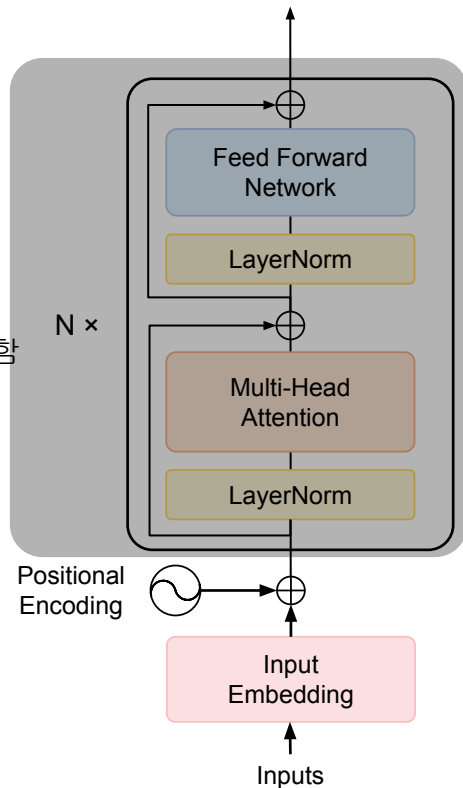
※ 단어 정보가 같더라도, 순서가 다르면 의미가 달라짐

따라서, 각 토큰 벡터에 위치 정보를 더해, 모델이 ‘어디에 있는지를’ 이해할 수 있도록 함



### Positional Encoding 을 어떻게 계산될까?

가장 기본적인 Positional Encoding 은  $\sin / \cos$  같은 주기함수를 여러 주파수로 조합해 만든다. (sinusoidal, RoPE)  
하나의 위치는 여러 파형에서 서로 다른 값을 가져, 위치마다 고유한 패턴이 만들어지며, 모델은 이 값을 보고 상대적/절대적 위치를 파악할 수 있다.  
또, learnable vector embedding 을 학습시키는 경우도 존재한다.

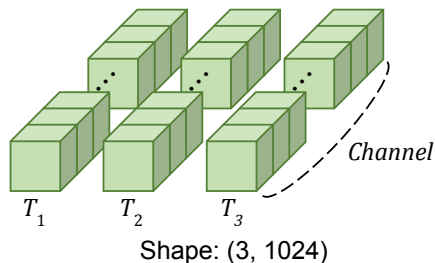


# Layer Normalization (LayerNorm)

## Layer Normalization 의 효과

✓ 각 토큰을 독립적으로 정규화 하여 벡터의 스케일을 안정화

✓ 학습을 더 안정적으로 도와줌

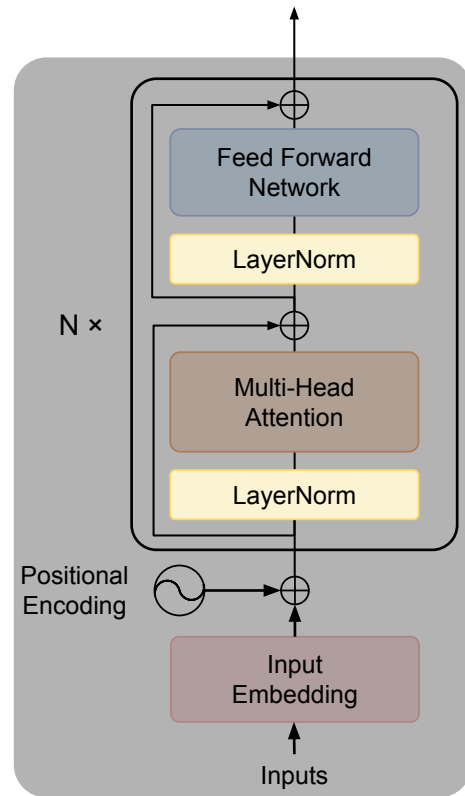


$$y_i = \gamma_i \left( \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}} \right) + \beta_i$$

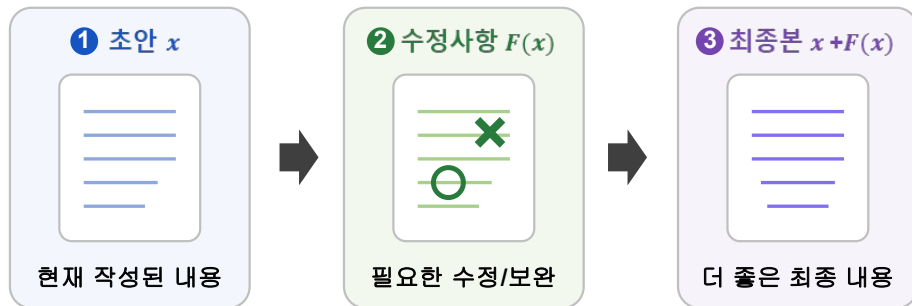
$\mu$ : Channel 값 평균  
 $\sigma^2$ : Channel 값 분산  
 $\epsilon$ : 분모가 0에 가까워지는 것을 방지하는 수치 안정화 상수  
 $\gamma$ : 정규화된 토큰의 Channel 별 scale parameter (learnable)  
 $\beta$ : 정규화된 토큰의 Channel 별 shift parameter (learnable)

| Token 1 ( $T_1$ )                            | Token 2 ( $T_2$ )                            | Token 3 ( $T_3$ )                            |
|----------------------------------------------|----------------------------------------------|----------------------------------------------|
| $[c_1, c_1, c_1, \dots, c_{1023}, c_{1024}]$ | $[c_1, c_1, c_1, \dots, c_{1023}, c_{1024}]$ | $[c_1, c_1, c_1, \dots, c_{1023}, c_{1024}]$ |
| 평균이 0이 아니고, 스케일(분산)이 클                       | 평균이 0이 아니고, 스케일(분산)이 클                       | 평균이 0이 아니고, 스케일(분산)이 클                       |

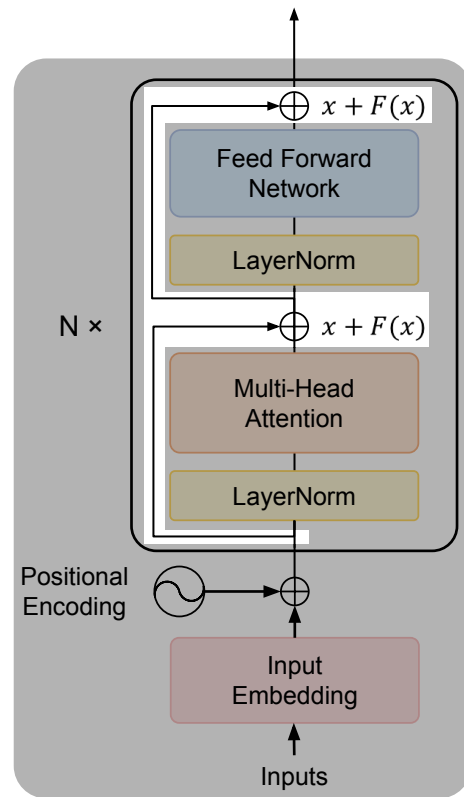
각 토큰별로 벡터의 안정화/정규화 (Normalization) 필요!



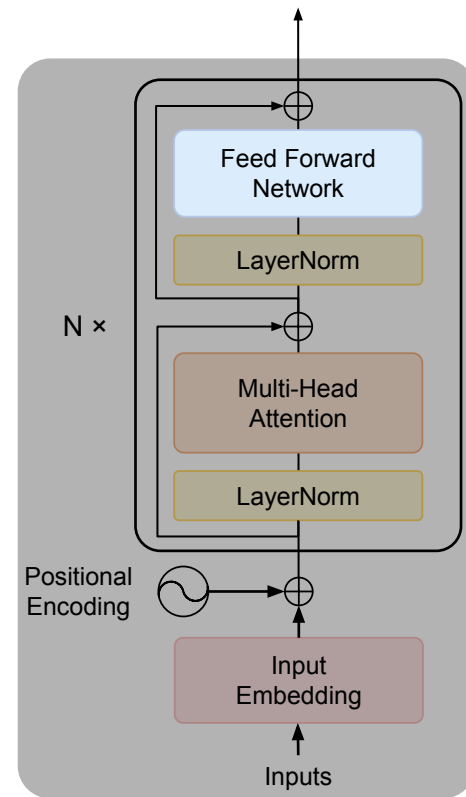
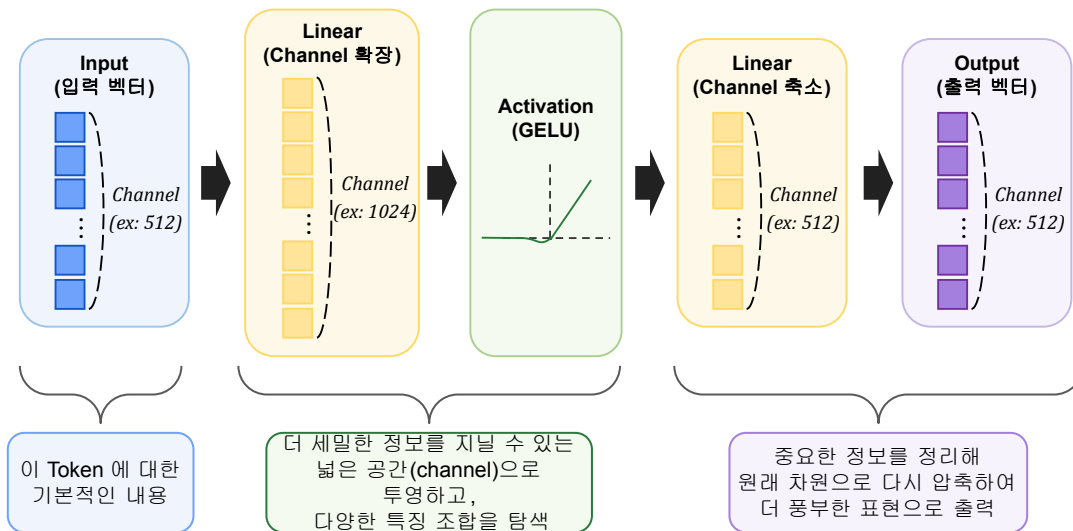
# Residual Connection



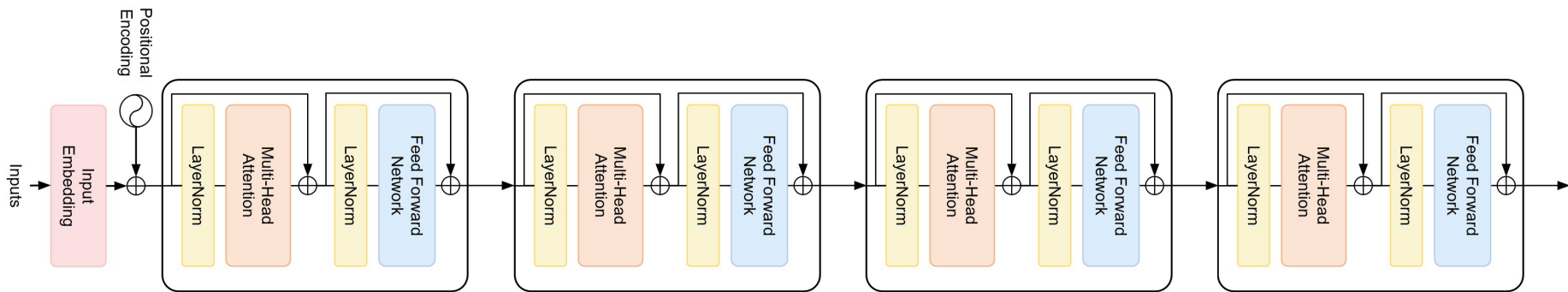
- ✓ 입력을 통째로 새로 만드는 대신, 필요한 변경량(잔차)  $F(x)$ 만 학습
- ✓ 기존 표현  $x$ 를 유지하면서, 필요한  $F(x)$ 만 더하여, 한 번에 전부 바꾸기 보다 조금씩 보정/수정
- ✓ 서브 레이어는 필요한 변화에만 집중할 수 있어 학습을 더 안정적으로 도와줌



# Feed Forward Network (FFN)



## From Block to Transformer (Repeat Block )



- ✓ 최종적으로 **block** 을 여러번 반복하여 **Transformer** 완성
- ✓ **layer** 가 깊을수록 더욱 큰 모델이며, 일반적으로 더욱 강력한 성능을 발휘함  
(더욱 풍부한 고수준의 표현을 학습)
- ✓ 하지만 **layer**가 깊어질수록 학습 비용과 연산량도 함께 증가함